

How Amazon Finance Automation built a generative AI Q&A chat assistant using Amazon Bedrock

by Soheb Moin, Nitin Arora, Yunfei Bai, Mohak Chugh, Kumar Satyen Gaurav, and Parth Bavishi | on 02 DEC 2024 | in [Amazon Bedrock](#), [Artificial Intelligence](#), [Best Practices](#), [Customer Solutions](#), [Generative AI](#), [Intermediate \(200\)](#) | [Permalink](#) | [Comments](#) | [Share](#)

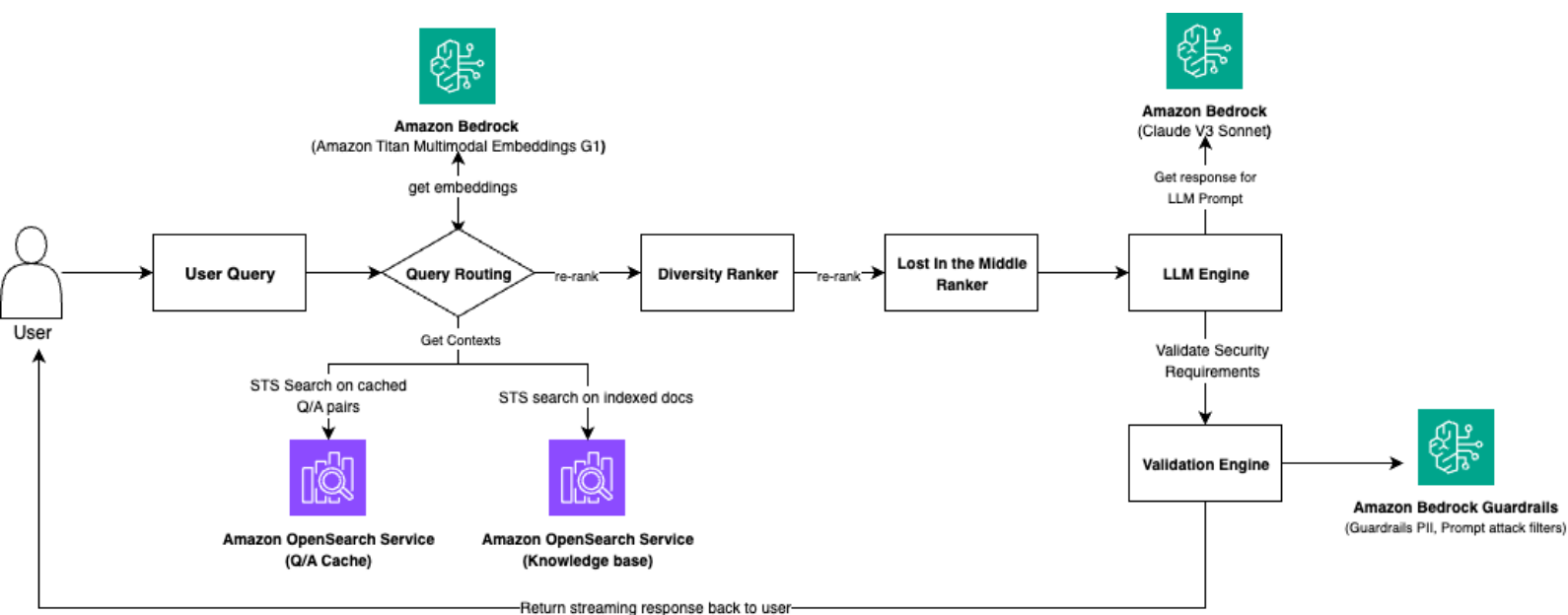
Today, the Accounts Payable (AP) and Accounts Receivable (AR) analysts in Amazon Finance operations receive queries from customers through email, cases, internal tools, or phone. When a query arises, analysts must engage in a time-consuming process of reaching out to subject matter experts (SMEs) and go through multiple policy documents containing standard operating procedures (SOPs) relevant to the query. This back-and-forth communication process often takes from hours to days, primarily because analysts, especially the new hires, don't have immediate access to the necessary information. They spend hours consulting SMEs and reviewing extensive policy documents.

To address this challenge, Amazon Finance Automation developed a [large language model](#) (LLM)-based question-answer chat assistant on [Amazon Bedrock](#). This solution empowers analysts to rapidly retrieve answers to customer queries, generating prompt responses within the same communication thread. As a result, it drastically reduces the time required to address customer queries.

In this post, we share how Amazon Finance Automation built this [generative AI](#) Q&A chat assistant using Amazon Bedrock.

Solution overview

The solution is based on a [Retrieval Augmented Generation](#) (RAG) pipeline running on Amazon Bedrock, as shown in the following diagram. When a user submits a query, RAG works by first retrieving relevant documents from a knowledge base, then generating a response with the LLM from the retrieved documents.



The solution consists of the following key components:

- Knowledge base** – We used [Amazon OpenSearch Service](#) as the vector store for embedding documents. For performance evaluation, we processed and indexed multiple Amazon finance policy documents into the knowledge base. Alternatively, [Amazon Bedrock Knowledge Bases](#) provides fully managed support for end-to-end RAG workflows. We're planning to migrate to Amazon Bedrock Knowledge Bases to eliminate cluster management and add extensibility to our pipeline.
- Embedding model** – At the time of writing, we're using the [Amazon Titan Multimodal Embeddings G1](#) model on Amazon Bedrock. The model is pre-trained on large and unique datasets and corpora from Amazon and provides accuracy that is

higher than or comparable to other embedding models on the market based on our comparative analysis.

3. **Generator model** – We used a [foundation model](#) (FM) provided by Amazon Bedrock for its balanced ability to deliver highly accurate answers quickly.
4. **Diversity ranker** – It's responsible for rearranging the results obtained from vector index to avoid skewness or bias towards any specific document or section.
5. **Lost in the middle ranker** – It's responsible for efficiently distributing the most relevant results towards the top and bottom of the prompt, maximizing the impact of the prompt's content.
6. **Guardrails** – We used [Amazon Bedrock Guardrails](#) to detect personal identifiable information (PII) and safeguard against prompt injection attacks.
7. **Validation engine** – Removes PII from the response and checks whether the generated answer aligns with the retrieved context. If not, it returns a hardcoded "I don't know" response to prevent hallucinations.
8. **Chat assistant UI** – We developed the UI using [Streamlit](#), an open source Python library for web-based application development on [machine learning](#) (ML) use cases.

Evaluate RAG performance

The accuracy of the chat assistant is the most critical performance metric to Amazon Finance Operations. After we built the first version of the chat assistant, we measured the bot response accuracy by submitting questions to the chat assistant. The SMEs manually evaluated the RAG responses one by one, and found only 49% of the responses were correct. This was far below the expectation, and the solution needed improvement.

However, manually evaluating the RAG isn't sustainable—it requires hours of effort from finance operations and engineering teams. Therefore, we adopted the following automated performance evaluation approach:

- **Prepare testing data** – We constructed a test dataset with three data fields:
 - **question** – This consists of 100 questions from policy documents where answers reside in a variety of sources, such as policy documents and engineering SOPs, covering complex text formats such as embedded tables and images.
 - **expected_answer** – These are manually labeled answers by Amazon Finance Operations SMEs.
 - **generated_answer** – This is the answer generated by the bot.
- **NLP scores** – We used a test dataset to calculate the [ROUGE](#) score and [METEOR](#) score. Because these scores merely use word-matching algorithms and ignore the semantic meaning of the text, they aren't aligned with the SME scores. Based on our analysis, the variance was approximately 30% compared to human evaluations.
- **LLM-based score** – We used an FM offered by Amazon Bedrock to score the RAG performance. We designed specialized LLM prompts to evaluate the RAG performance by comparing the generated answer with the expected answer. We generated a set of LLM-based metrics, including accuracy, acceptability, and factualness, and the citation representing the evaluation reasoning. The variance of this approach was approximately 5% compared to human analysis, so we decided to stick to this approach of evaluation. If your RAG system is built on Amazon Bedrock Knowledge Bases, you can use the new [RAG evaluation for Amazon Bedrock Knowledge Bases](#) tool to evaluate the retrieve or the retrieve and generate functionality with an LLM as a judge. It provides retrieval evaluation metrics such as context relevance and context coverage. It also provides retrieve and generate evaluation metrics such as correctness, completeness, and helpfulness, as well as responsible AI metrics such as harmfulness and answer refusal.

Improve the accuracy of RAG pipeline

Based on the aforementioned evaluation techniques, we focused on the following areas in the RAG pipeline to improve the overall accuracy.

Add document semantic chunking to improve accuracy from 49% to 64%

Upon diagnosing incorrect responses in the RAG pipeline, we identified 14% of the inaccuracy was due to incomplete contexts sent to the LLM. These incomplete contexts were originally generated by the segmentation algorithm based on a fixed chunk size (for example, 512 tokens or 384 words), which doesn't consider document boundaries such as sections and paragraphs.

To address this problem, we designed a new document segmentation approach using QUILL Editor, Amazon Titan Text Embeddings, and OpenSearch Service, using the following steps:

1. Convert the unstructured text to a structured HTML document using QUILL Editor. In this way, the HTML document preserves the document formatting that divides the contents into logical chunks.
2. Identify the logical structure of the HTML document and insert divider strings based on HTML tags for document segmentation.
3. Use an embedding model to generate semantic vector representation of document chunks.
4. Assign tags based on important keywords in the section to identify the logical boundaries between sections.
5. Insert the embedding vectors of the segmented documents to the OpenSearch Service vector store.

The following diagram illustrates the document retriever splitting workflow.

When processing the document, we follow specific rules:

- Extract the start and end of a section of a document precisely
- Extract the titles of the section and pair them with section content accurately
- Assign tags based on important keywords from the sections
- Persist the markdown information from the policy while indexing
- Exclude images and tables from the processing in the initial release

With this approach, we can improve RAG accuracy from 49% to 64%.

Use prompt engineering to improve accuracy from 64% to 76%

Prompt engineering is a crucial technique to improve the performance of LLMs. We learned from our project that there is no one-size-fits-all prompt engineering approach; it's a best practice to design task-specific prompts. We adopted the following approach to enhance the effectiveness of the prompt-to-RAG generator:

- In approximately 14% of cases, we identified that the LLM generated responses even when no relevant context was retrieved from the RAG, leading to hallucinations. In this case, we engineered prompts and asked the LLM not to generate any response when there is no relevant context provided.
- In approximately 13% of cases, we received user feedback that the response from the LLM was too brief, lacking complete context. We engineered prompts that encouraged the LLM to be more comprehensive.
- We engineered prompts to enable the capability to generate both concise and detailed answers for the users.

- We used LLM prompts for generation of citations to properly attribute our source used to generate the answer. In the UI, the citations are listed with hyperlinks following the LLM response, and users can use these citations to validate the LLM performance.
- We improved our prompts to introduce better chain-of-thought (CoT) reasoning:
 - The LLM's unique characteristic of using internally generated reasoning contributes to improved performance and aligns responses with humanlike coherence. Because of this interplay between prompt quality, reasoning requests, and the model's inherent capabilities, we could optimize performance.
 - Encouraging CoT reasoning prompts the LLM to consider the context of the conversation, making it less prone to hallucinations.
 - By building upon the established context, the model is more likely to generate responses that logically follow the conversation's narrative, reducing the chances of providing inaccurate or hallucinated answers.
 - We added examples of previously answered questions to establish a pattern for the LLM, encouraging CoT.

We then used meta-prompting using an FM offered by Amazon Bedrock to craft a prompt that caters to the aforementioned requirements.

The following example is a prompt for generating a quick summary and a detailed answer:

```
You are an AI assistant that helps answer questions based on provided text context. I will give you

<context>
{}
</context>

And here is the question:

<question>
{}
</question>

Think carefully about how the context can be used to answer the question.

<thinkingprocess>
- Carefully read the provided context and analyze what information it contains
- Identify the key pieces of information in the context that are relevant to answering the question
- Determine if the context provides enough information to answer the question satisfactorily
- If not, simply state "I don't know, I don't have the complete context needed to answer this question"
```

The following example is a prompt for generating citations based on the generated answers and retrieved contexts:

```
You are an AI assistant that specializes in attributing generated answers to specific sections with

Here is the generated answer to analyze:

<generated_answer>
{}
</generated_answer>

And here are the sections from various documents to consider:
```

```
<sections>
{}
</sections>
```

```
Please carefully read through the generated answer and the provided sections. In the scratchpad space, you can write down your thoughts.

<scratchpad>
</scratchpad>
```

After identifying the relevant sections, provide your output in the following format:

By implementing the prompt engineering approaches, we improved RAG accuracy from 64% to 76%.

Use an Amazon Titan Text Embeddings model to improve accuracy from 76% to 86%

After implementing the document segmentation approach, we still saw lower relevance scores for retrieved contexts (55–65%), and the incorrect contexts were in the top ranks for more than 50% of cases. This indicated that there was still room for improvement.

embedding models such as bge-base-en-v1.5 performed better for context retrieval, comparing to other top embedding models such as all-mpnet-base-v2. We found that using the Amazon Titan Embeddings G1 model increased the possibility of retrieved contexts from approximately 55–65% to 75–80%, and 80% of the retrieved contexts have higher ranks than before.

Finally, by adopting the [Amazon Titan Text Embeddings G1](#) model, we improved the overall accuracy from 76% to 86%.

Conclusion

We achieved remarkable progress in developing a generative AI Q&A chat assistant for Amazon Finance Automation by using a RAG pipeline and LLMs on Amazon Bedrock. Through continual evaluation and iterative improvement, we have addressed challenges of hallucinations, document ingestion issues, and context retrieval inaccuracies. Our results have shown a significant improvement in RAG accuracy from 49% to 86%.

You can follow our journey and adopt a similar solution to address challenges in your RAG application and improve overall performance.

About the Authors

Soheb Moin is a Software Development Engineer at Amazon, who led the development of the Generative AI chatbot. He specializes in leveraging generative AI and Big Data analytics to design, develop, and implement secure, scalable, innovative solutions that empowers Finance Operations with better productivity, automation. Outside of work, Soheb enjoys traveling, playing badminton, and engaging in chess tournaments.

Nitin Arora is a Sr. Software Development Manager for Finance Automation in Amazon. He has over 19 years of experience building business critical, scalable, high-performance software. Nitin leads data services, communication, work management and several Generative AI initiatives within Finance. In his spare time, he enjoys listening to music and read.

Yunfei Bai is a Principal Solutions Architect at AWS. With a background in AI/ML, data science, and analytics, Yunfei helps customers adopt AWS services to deliver business results. He designs AI/ML and data analytics solutions that overcome

complex technical challenges and drive strategic objectives. Yunfei has a PhD in Electronic and Electrical Engineering. Outside of work, Yunfei enjoys reading and music.

Kumar Satyen Gaurav is an experienced Software Development Manager at Amazon, with over 16 years of expertise in big data analytics and software development. He leads a team of engineers to build products and services using AWS big data technologies, for providing key business insights for Amazon Finance Operations across diverse business verticals. Beyond work, he finds joy in reading, traveling and learning strategic challenges of chess.

Mohak Chugh is a Software Development Engineer at Amazon, with over 3 years of experience in developing products leveraging Generative AI and Big Data on AWS. His work encompasses a range of areas, including RAG based GenAI chatbots and high performance data reconciliation. Beyond work, he finds joy in playing the piano and performing with his music band.

Parth Bavishi is a Senior Product Manager at Amazon with over 10 years of experience in building impactful products. He currently leads the development of generative AI capabilities for Amazon's Finance Automation, driving innovation and efficiency within the organization. A dedicated mentor, Parth enjoys sharing his product management knowledge and finds satisfaction in activities like volleyball and reading.

 Like (2)  Share

Comments

Log in to comment
